High-Level Design (HLD)

RAG-Based Customer Support Assistant

1. System Overview

🔹 Problem Definition

Traditional customer support systems often provide inaccurate or hallucinated responses. There is a need for a system that generates **reliable, context-aware answers strictly from a knowledge base**.

🔹 Solution

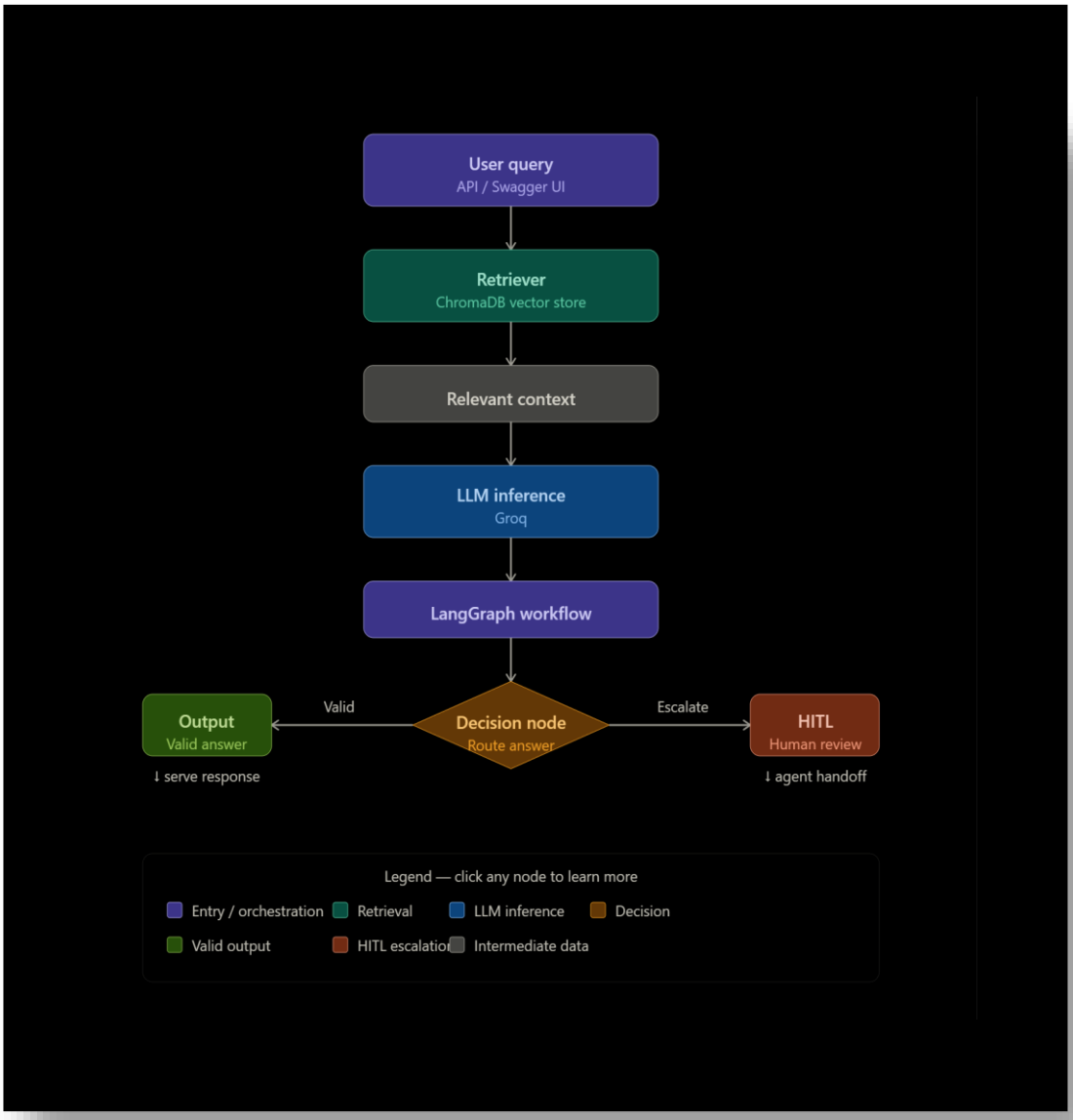
This project implements a **Retrieval-Augmented Generation (RAG)** system that:

- Processes a PDF knowledge base
- Retrieves relevant information using embeddings
- Generates grounded responses using an LLM
- Uses a graph-based workflow (LangGraph)
- Escalates to human (HITL) when needed

🔹 Scope

- Single PDF knowledge base
- FAQ-style customer support queries
- Backend API-based interaction (FastAPI)

2. Architecture Diagram



3. Component Description

🔹 Document Loader

- Loads PDF using PyPDFLoader
- Extracts raw text content

🔹 Chunking Strategy

- Custom **Q&A-based chunking**
- Ensures each chunk contains both question and answer
- Prevents incomplete retrieval

- **Embedding Model**
  - HuggingFace model (all-MiniLM-L6-v2)
  - Converts text into vector embeddings

- **Vector Store**
  - ChromaDB used for storing embeddings
  - Enables semantic similarity search

- **Retriever**
  - Fetches top-k relevant chunks based on query
  - Uses vector similarity

- **LLM (Groq)**
  - Generates responses using retrieved context
  - Strictly constrained to avoid hallucination

- **Graph Workflow Engine (LangGraph)**
  - Controls execution flow
  - Nodes:
    - Process Node
    - Decision Node
    - Output Node
    - HITL Node

- **Routing Layer**
  - Decides:
    - Return answer
    - Escalate to human

- **HITL Module**
  - Handles escalation when:
    - No context
    - Low confidence
    - Unknown query

4. Data Flow

- **Ingestion Flow**

PDF → Load → Q&A Chunking → Embeddings → ChromaDB

- **Query Flow**

User Query → Retriever → Context → LLM → Decision → Output / HITL

- **Query Lifecycle**
  1. User sends query
  2. Retriever fetches relevant chunk
  3. Context passed to LLM
  4. LLM generates answer
  5. Decision node evaluates
  6. Output or escalation

5. Technology Choices

- **ChromaDB**
  - Lightweight
  - Fast similarity search
  - Easy local setup

- **LangGraph**
  - Explicit workflow control
  - Supports conditional routing

- Better than linear pipelines

🔹 **Groq LLM**

- Low latency
- High performance inference
- Suitable for real-time systems

🔹 **FastAPI**

- Fast and simple API framework
- Built-in Swagger UI

6. Scalability Considerations

🔹 **Large Documents**

- Can split into multiple PDFs
- Efficient chunking strategy

🔹 **High Query Load**

- Use caching
- Deploy with async workers

🔹 **Latency**

- Groq reduces inference latency
- Vector search optimized via Chroma

🔹 **Future Scaling**

- Multi-document support
- Distributed vector DB
- Load balancing